Reinforcement Learning for a Pokémon Nuzlocke Challenge

DSPRO2 Final Report

Tristan Muri Marvin Hueber Viacheslav Godovskii Alex Liubov

Spring Semester 2026

Abstract

This report documents the development of a reinforcement learning agent trained to complete a Pokémon Nuzlocke challenge: a sequential battle gauntlet where fainted Pokémon are permanently lost and a single defeat ends the run. The system uses PPO (Proximal Policy Optimization) with a transformer-based policy network, trained through curriculum learning against progressively stronger opponents. We describe the full engineering lifecycle: environment integration with the Pokémon Showdown battle simulator, observation pipeline design, model architecture, distributed training infrastructure, systematic debugging of the observation and reward pipeline, and an honest analysis of training results. The report demonstrates an end-to-end, reproducible machine learning workflow aligned with MLOps best practices, including experiment tracking via MLflow, checkpoint management, and a validation suite.

Contents

+

1. Introduction

1.1 Problem Statement

The goal of this project is to develop an AI agent capable of completing a *Pokémon Nuzlocke challenge* using deep reinforcement learning. The chosen game is *Pokémon Brilliant Diamond* (BDSP), and the agent must defeat all mandatory trainers without losing, subject to constraints that transform standard battles into a complex sequential decision-making problem under uncertainty with irreversible consequences.

A Nuzlocke challenge is a self-imposed difficulty rule set widely adopted by the Pokémon community. Its core constraints are:

- **First-catch rule:** Only the first Pokémon encountered in each new location may be captured.
- **Permadeath:** If a Pokémon faints during battle, it is considered permanently lost and cannot be used again.
- **Run failure:** If the player loses a battle with no usable Pokémon remaining, the entire run ends and must restart.

These rules introduce permanent consequences and limited resources, turning a turn-based strategy game into a problem that requires long-horizon planning, risk assessment, and resource management, all qualities that make it a compelling testbed for reinforcement learning.

1.2 Stakeholder Context

This project is carried out as part of the **DSPRO2** course at the Hochschule Luzern (HSLU) during the Spring 2026 semester. Beyond fulfilling academic requirements, the project serves as a *reusable experimental benchmark* for evaluating reinforcement learning algorithms on generalisation, robustness, and risk-aware decision-making.

The primary stakeholders are:

- **Course supervisors:** Expect a professional-grade report demonstrating an end-to-end ML workflow, experiment tracking, honest validation, and actionable conclusions.
- **Project team:** Uses the system as a portfolio-quality technical artifact proving the ability to move from prototype to a production-ready mindset.
- **RL research community:** The Nuzlocke benchmark, if successful, provides a challenging, well-defined testbed for comparing RL approaches on tasks with sparse rewards, partial observability, and irreversible actions.

1.3 Objectives & Success Metrics

The overarching objective is to train an AI agent that successfully completes a Nuzlocke gauntlet: a predefined sequence of battles against trainer teams from Pokémon Brilliant Diamond.

Success is measured by the following metrics:

Metric	Description
Gauntlet completion rate	Fraction of runs where the agent beats all trainers in the correct sequence.
Progression depth	Average number of battles completed before failure.
Remaining resources	HP and Pokémon still available upon gauntlet completion.
Generalisation	Performance on unseen gauntlet orderings or opponent teams without further training.
Model efficiency	Minimum model size (parameters) needed to reach a given competency level.

Near-term success is defined as achieving a stable win rate above 50 % against heuristic opponents in the mixed-final curriculum stage (see Chapter 3).

1.4 Project Scope

The project focuses exclusively on **battle decision-making**:

- Move selection and Pokémon switching during battles.
- Team composition (in later milestones).
- Sequential gauntlet management with Nuzlocke constraints.

The following are **explicitly excluded**:

- Game-world exploration and traversal.
- Interaction with an actual game client: the system operates entirely through the Pokémon Showdown battle simulator.
- Multiplayer or competitive battling against human players.

1.5 Document Structure

This report is organised as follows. Chapter 2 provides background on Pokémon battle mechanics, the simulation environment, the challenge model, data provenance, and the milestone roadmap. Chapter 3 describes the complete methodology: the reinforcement learning formulation, model architecture, training infrastructure, and the iterative development process through which the pipeline was debugged and improved. Chapter 4 presents training results for both fixed-team and random-team settings, including a chronological development progression and an honest assessment. Chapter 5 discusses risk, ethical, and human-in-the-loop considerations. Chapter 6 concludes with lessons learned and next steps. Appendix A provides a complete reproduction guide.

2. Background

2.1 Pokémon Battles: Overview

Pokémon battles are turn-based strategy matches. Each player brings a team of up to six Pokémon, but usually only one is active at a time (singles format). The objective is to reduce all opposing Pokémon to zero HP (fainted).

2.1.1 Turn Structure

At the start of each turn, both players simultaneously choose a hidden action:

- **Attack:** Use one of the active Pokémon's four available moves.
- **Switch:** Replace the active Pokémon with another from the bench. Switching happens instantly before any moves are executed.

If both players choose to attack, execution order is determined by:

1. **Move priority:** Certain moves (e.g. Quick Attack, Protect) always go first regardless of speed.
2. **Speed stat:** Among equal-priority moves, the faster Pokémon acts first.

2.1.2 Pokémon Anatomy

Each Pokémon possesses:

Attribute	Description
Types (1–2)	Determine offensive strengths/weaknesses and defensive resistances.
Four moves	Damaging attacks or status-altering moves with varying power, accuracy, and effects.
Ability	A passive effect that influences the battle (e.g. Intimidate lowers the opponent's Attack on entry).
Held item	A consumable or passive item that boosts stats, heals, or triggers effects.
Base stats (6)	HP, Attack, Defense, Sp. Atk, Sp. Def, Speed: determine overall power and role.

2.1.3 Damage & Typing

Damage is calculated from Attack vs. Defence stats, modified by type interactions:

Modifier	Multiplier
STAB (Same-Type Attack Bonus)	$\times 1.5$
Super Effective	$\times 2.0$
Resisted	$\times 0.5$
Immune	$\times 0.0$

These modifiers stack multiplicatively, so a doubly super-effective STAB move deals $1.5 \times 2.0 \times 2.0 = 6\times$ base damage. Understanding type matchups is therefore central to competent play.

2.1.4 Winning & Losing

When a Pokémon's HP reaches zero, it faints. The owning player must send out their next available Pokémon. The battle ends when one side has no remaining conscious Pokémon. In the Nuzlocke variant, fainted Pokémon are permanently unavailable for subsequent battles.

2.2 Pokémon Showdown Simulator

The project does not interact with an actual game client. Instead, it uses **Pokémon Showdown**: an open-source battle simulator that implements official Pokémon battle mechanics with high fidelity. The simulator runs as a local Node.js server, enabling fast, programmatic battle execution.

Key properties:

- Supports multiple generations (the project uses **Gen 8** / `gen8randombattle` format).
- Runs as a WebSocket server, accepting connections from automated clients.
- Handles all battle mechanics internally: damage rolls, stat stages, weather, terrain, status conditions, abilities, and items.
- Provides structured protocol messages describing battle state changes each turn.

For distributed training, **eight parallel Showdown instances** are spun up on ports 8000–8007 (see Section 3.3).

2.3 poke-env Python Library

`poke-env` provides the Python interface to Pokémon Showdown:

- A structured API for interacting with battles (reading state, choosing actions).
- Battle state objects that expose team composition, active Pokémon, moves, HP, status effects, weather, and more.
- Built-in player classes including `RandomPlayer` (uniform random over legal orders) and `SimpleHeuristicsPlayer` (rule-based AI considering type effectiveness and switching).
- Integration utilities for reinforcement learning environments.

The project extends `poke-env` with a custom player class (`RandomNoSwitchPlayer`) that samples uniformly among legal *move* orders and never voluntarily switches, useful as a baseline opponent that forces the agent to learn move selection before switching strategy.

2.4 Pokémon Brilliant Diamond Context

The target game is Pokémon Brilliant Diamond (BDSP), a remake of the Generation 4 titles. The choice is motivated by:

- A well-documented set of mandatory trainer battles.
- Available datasets of trainer teams with exact Pokémon, levels, movesets, and items (see Section 2.5).
- Gen 8 mechanics in Showdown provide a close match to BDSP's battle system.

The gauntlet (the ordered sequence of battles the agent must complete) comprises 706 trainers extracted from BDSP gameplay data, ordered by ascending difficulty (level, team size).

2.5 Challenge Definition

Instead of playing through the entire Pokémon game world, the project focuses on **battle sequences**. The challenge is modelled as a graph of battles: each node represents a battle against a specific trainer team, and edges represent progression.

In the simplest form (currently implemented), the challenge is a linear list of opponents. The agent must beat all trainers in order, without losing. If the agent loses a battle, the run ends and restarts from the beginning.

2.5.1 Resource Management Between Battles

After each battle, the agent's resources are *partially* restored:

- HP and status conditions are fully healed.
- Power Points (PP, move usage limits) are restored.
- **Fainted Pokémon remain lost** (Nuzlocke rule).

Usually, the agent may build a new team before each fight. There are exceptions where the agent must battle through a sequence of opponents (up to five) before being allowed to change teams again. The amount of information available to the team builder (complete, partial, or none) serves as a knob for testing agent competency.

2.6 Milestone Roadmap

Due to the large scope of the project, four progressive milestones have been defined. For each milestone, a model is trained from scratch and evaluated against the gauntlet to validate competency.

#	Milestone	Goal
1	Proof of Concept	The agent uses a *fixed team* (6 Pokémon, reused every battle) against *random opponents*. Tests whether the model can learn to use a known team against unseen opponents.
2	Random Battles	Both the agent and opponent use *randomly generated teams*. Tests competency under high entropy, where each battle is novel.
3	Team Building	Model 1 learns to compose optimal teams given opponent information; Model 2 learns to battle with those teams. The team builder must construct a team that can beat at least five different opponents.
4	Gauntlet Master	Full gauntlet completion under Nuzlocke constraints. The ultimate objective.

Current status: Milestone 2 is in progress. The training infrastructure is fully functional and multiple architectural improvements have been validated (see Section 3.4), but random team competency has not yet been demonstrated.

2.7 Data Provenance

All data used in this project is derived from publicly available sources or generated programmatically. No personal data is involved.

2.7.1 BDSP Trainer Dataset

File	Format	Content
`data/BDSP Trainer Data - v1.1.csv`	CSV	Complete roster of 706 BDSP trainers with Pokémon species, levels, movesets, held items, and abilities.
`data/bdsp_gauntlet_order.json`	JSON	Ordered list of 706 trainers defining the gauntlet sequence, sorted by ascending difficulty (level, team size). Includes metadata about breakpoints and battle counts.
`data/bdsp_trainers.json`	JSON	Trainer team specifications in Showdown-compatible format. Used to instantiate specific opponents during training and evaluation.

Gauntlet evaluation against this sequence is still **future work**; the files are kept for reproducibility and planned Milestone 4 integration.

2.7.2 Vocabulary Files

Categorical entities (species, items, abilities) are mapped to integer IDs via stable dictionary lookups stored in JSON:

Entity	Vocabulary Size	Source
Species	1 515	All Gen 8 species (dictionary vocab)
Items	583	All available held items
Abilities	314	All available abilities

These vocabularies were generated programmatically from Pokémon Showdown's data files and validated for complete coverage over the BDSP trainer dataset and validation manifests (zero unknown entities).

2.7.3 Validation Team Manifests

For reproducible evaluation, a set of frozen team configurations is maintained:

- `data/validation/gen8_random_battle_team_pairs.json`: 20 teams organised as 10 swapped pairs, yielding 20 battles per opponent AI per evaluation run.
- `data/validation/gen8_random_mirror_teams.json`: Mirror-matchup teams for symmetric evaluation.

These teams are evaluated against both `RandomPlayer` and `SimpleHeuristicsPlayer` opponents, providing a fixed benchmark for comparing checkpoints.

2.8 Data Preprocessing Pipeline

Raw data undergoes the following transformations before reaching the agent:

1. **Trainer data loading:** CSV is parsed into an internal representation; team JSON files are loaded and converted to Showdown-format team strings.
2. **Team generation:** For random battles, teams are sampled from the Gen 8 random battle pool with seed control for reproducibility.

3. **Observation construction:** Each turn, the raw battle state from `poke-env` is encoded into a fixed-size tensor (13×164 floats) plus categorical ID vectors for species, items, and abilities. The full encoding pipeline is detailed in Section 3.2 and Appendix B.
4. **Action mapping:** The agent's compressed 14-action output is mapped to native `poke-env` `BattleOrder` objects at runtime (see Section 3.2.6).

3. Methods

This chapter describes the reinforcement learning framework, model architecture, training infrastructure, and the iterative development methodology used to diagnose and fix issues in the training pipeline.

3.1 Reinforcement Learning Framework

3.1.1 MDP Formulation

The Pokémon battle environment is modelled as a Markov Decision Process (MDP) with the following components:

Component	Definition
State (s)	Dictionary observation: token matrix (13×164), species/item/ability ID vectors $(13,)$ each, action mask $(14,)$. Full specification in Section 3.2.
Action (a)	Discrete choice from 14 possible actions (4 moves, 4 gimmick-enhanced moves, 6 switches), filtered by the action mask to only legal options.
Reward (r)	Shaped reward computed each step from HP changes, fainting events, and a terminal outcome bonus/penalty. Detailed in Section 3.1.3.
Transition	Deterministic given both players' actions; stochasticity arises from the opponent's policy and damage variance in the simulator.
Discount (γ)	0.97 based on a hyper parameter sweep.

An episode corresponds to a single battle. It terminates when one side has no remaining conscious Pokémon (win or loss). There is no explicit time limit or truncation.

3.1.2 PPO Algorithm

The agent is trained with **Proximal Policy Optimization** (PPO), an on-policy actor-critic algorithm that balances sample efficiency with training stability through a clipped surrogate objective. PPO was chosen for the following reasons:

- **Stability:** The clipped objective prevents destructively large policy updates, which is critical when learning in a sparse-reward, high-variance environment.
- **Distributed rollouts:** PPO naturally integrates with Ray RLlib's distributed rollout architecture, enabling many parallel battles.
- **Action masking:** PPO's categorical distribution can be combined with action masking (setting invalid-action logits to a large negative value) without modifying the algorithm itself.

The policy network (actor) outputs a categorical distribution over the 14 actions, and the value network (critic) outputs a scalar state-value estimate $V(s)$. Both share a common transformer trunk (see Section 3.2).

3.1.3 Reward Design

The reward function combines several components to provide dense learning signal while preserving the primacy of winning. All rewards are multiplied by a global `reward_scale` $= 0.1$ or 0.05 before being returned to the agent, which scales returns from approximately $[-15, +15]$ to $[-0.75, +0.75]$. The rationale for this scaling is discussed in Section 3.4.10.

Component	Default	Description
Victory reward	+10.0	Agent wins the battle (terminal step).
Defeat penalty	−10.0	Agent loses the battle (terminal step).
HP differential	×2.0	(our_team_HP−opp_team_HP)×w as a shaped step reward. Positive when the agent is ahead, negative when behind.
Opponent fainted	+5.0	Per opponent Pokémon knocked out.
Own fainted	−5.0	Per agent Pokémon that faints.
Step penalty	−0.005	Per step, encourages battle efficiency (fewer turns).
Matchup quality	×0.2	+1.0 if a super-effective move is available, −0.5 if only resisted moves, −1.0 if immune. Rewards good type-readiness.
Action quality	×0.3	Penalty for clearly suboptimal moves; bonus for resisting the opponent's best move.

The reward is computed each step by `poke-env`'s `reward_computing_helper`, parameterised by the active `RewardConfig`. At the terminal step, explicit outcome rewards are applied on top. Reward parameters are adjusted across curriculum stages (see below). These were gradually introduced during development in descending order.

3.1.4 Curriculum Learning

Due to the complexity of the full challenge, initially not improving when just playing against heuristics, training follows a **curriculum learning** approach where the agent is first exposed to easy opponents and gradually faces stronger ones. The curriculum currently has four stages:

1. **Moves & Switches Warmup.** Opponent mix: 40 % random, 30 % `random_no_switch`, 30 % self-play. Promotion threshold: 65 % win rate. Asymmetric rewards (victory +8, defeat −10) encourage caution. Higher HP weight (3.0) and faint values (±6.0) provide strong per-step signal for learning basic move selection.
2. **Random, More Moves.** Opponent mix: 70 % self-play, 10 % random, 20 % `random_no_switch`. Promotion threshold: 20 % win rate (intentionally low: the goal is exposure to self-play, not mastery). Victory reward raised to +10. Large minimum sample requirement (5 000 episodes) ensures sufficient exposure before progressing.
3. **Self-Play.** Opponent mix: 60 % `random_no_switch`, 40 % random. Promotion threshold: 70 % win rate. After the self-play exposure stage, the agent trains against non-switching random opponents to solidify move selection skills. Action quality weight increased to 0.4 to refine move choice.
4. **Mixed Final.** Opponent mix: 50 % heuristic, 30 % self-play, 20 % `random_no_switch`. Terminal stage (promotion threshold 101 %, i.e. never promotes). The agent faces the strongest opponent mix, including the heuristic AI which considers type effectiveness and switches strategically.

3.1.5 Team-Pool Curriculum (Later Experiments)

After the curriculum above was established, a separate training line was added for **fixed team pools** on `gen8customgame` (preset `pure_league_play`). It does not replace the earlier four-stage setup; it extends the same PPO stack with staged opponent teams ($3 \to 5 \to 10 \to 20$) and a final `league_training` mix (roughly 68 % heuristic, 10 % self-play, 5 % historical snapshots, 12 % random, 5 % `random_no_switch`). Stages include `warmup`, `heuristic_bridge`, pool tactics stages, then league. Manifests live under `data/teams/pool_curriculum/`.

Per-Stage Reward Variants

Reward parameters are adjusted across stages to reflect changing difficulty:

Parameter	Stage 1	Stage 2	Stage 3	Stage 4
Victory / Defeat	+8 / −10	±10	±10	±10
HP weight	3.0	3.0	3.0	3.0
Opp. fainted	+6.0	+6.0	+6.0	+6.0
Own fainted	−6.0	−6.0	−6.0	−6.0
Step penalty	−0.01	−0.01	−0.01	−0.01
Matchup weight	0.1	0.1	0.1	0.1
Action quality wt.	0.3	0.3	0.4	0.4

Note that Stage 1 uses asymmetric rewards (victory +8 vs. defeat −10) to discourage early aggression, and a higher step penalty (−0.01) to encourage efficient play from the start.

Promotion decisions use a rolling window of the last 300 episodes (minimum 50 samples required before promotion is considered; minimum 300 episodes must elapse before any promotion). Total possible rewards is by design made in a way that can only increase with difficulty.

3.1.6 Self-Play Mechanism

Self-play is implemented through a `SelfPlayPlayer` class that:

1. Loads the latest model weights from `checkpoints/selfplay_latest.pt` on first use.
2. Re-checks the weights file each turn, enabling automatic updates when new checkpoints are saved.
3. Maintains per-battle LSTM state for temporal memory.
4. Converts the agent's compressed 14-action output to native `poke-env` `BattleOrder` objects.
5. Falls back to `RandomPlayer` behaviour on inference errors.

At each checkpoint, the training pipeline exports the current model weights to the self-play weights path, making them immediately available to all `SelfPlayPlayer` instances across workers. This creates a natural curriculum where the agent continuously adapts to an opponent of increasing skill. In later runs paths are resolved to absolute locations for Ray workers, and action selection uses temperature sampling ($\tau \approx 0.8$) instead of deterministic argmax.

3.1.7 Opponent Types

The environment supports four opponent categories:

Key	Player Class	Behaviour
`random_no_switch`	`RandomNoSwitchPlayer`	Uniform random among legal *move* orders; switches only when forced (no moves available or forced by simulator).
`random`	`RandomPlayer`	Uniform random over all legal orders (moves *and* switches).
`heuristic`	`SimpleHeuristicsPlayer`	Rule-based AI: considers type effectiveness, switches when at a disadvantage.
`self`	`SelfPlayPlayer`	Uses the training model's weights for inference with hot-reload support.
`historical`	Historical snapshot player	Samples past checkpoints from a small ring buffer (league stage only).

In the embedding layer, `random_no_switch` is mapped to the same bucket as `random` for the opponent-type feature, avoiding distribution shift at the observation level when mixing or swapping baselines. From best to worst in terms of performance: heuristic, random no switch and random.

3.2 Model Architecture

3.2.1 Overview

The system consists of three major components: (1) a **custom Gymnasium environment** wrapping `poke-env` and Pokémon Showdown, (2) a **transformer-based neural network** that processes battle observations and outputs action logits plus a state-value estimate, and (3) a **distributed training pipeline** built on Ray RLlib. This section details the first two; the training infrastructure is covered in Section 3.3.

3.2.2 Observation Space

The observation is a dictionary with five keys:

Key	Shape / Dtype	Content
`obs`	(13, 164) float32	Token matrix: 1 global token + 6 own-team tokens + 6 opponent-team tokens. Each token encodes presence flags, HP, base stats, types, status, moves, and more (see Appendix B).
`species`	(13,) int32	Species ID per token (dictionary lookup, vocabulary size 1 151).
`items`	(13,) int32	Held-item ID per token (vocabulary size 1 110).
`abilities`	(13,) int32	Ability ID per token (vocabulary size 216).
`action_mask`	(14,) float32	Binary mask: 1.0 for valid actions, 0.0 for invalid.

Token Layout

The 13 tokens are organised as follows:

Index	Role	Content
0	Global	Weather, terrain, side conditions, battle extras
1	Our active	Active Pokémon on the agent's side
2–6	Our bench	Remaining team Pokémon
7	Opp. active	Active opponent Pokémon
8–12	Opp. bench	Remaining opponent team

Each Pokémon token is 164 dimensions. The full per-token feature breakdown is provided in Appendix B.

3.2.3 Embedding Layers

High-cardinality categorical entities are mapped to learned embeddings:

Entity	Vocabulary Size	Embedding Dim
Species	1 515	32
Items	583	32
Abilities	314	32

Padding index 0 represents absent entities (e.g. a bench slot with no Pokémon). Each token's base 164-dim vector is concatenated with three 32-dim embedding vectors ($3 \times 32 = 96$), yielding 260 dimensions total. A linear projection followed by LayerNorm maps this to the transformer's hidden dimension (512 by default).

From Hashes to Dictionaries

Earlier versions used Adler-32 hashing with modulo 20 000 to compress entity strings into integer IDs. This caused approximately 7.5 % collision rate among 1 800 unique strings. The current system uses stable dictionary lookups generated from Showdown's data files, eliminating collisions entirely and ensuring that each entity receives a unique embedding.

3.2.4 Position & Role Embeddings

The transformer processes a set of 13 tokens. Without positional information, self-attention is permutation-invariant, meaning the model cannot distinguish between active and bench Pokémon. Two sets of learnable embeddings address this:

- **Position embeddings:** One learned vector per token position (13 positions → hidden dim). Added after input projection.
- **Role embeddings:** Five coarse roles: CLS (global), our active, our bench, opponent active, opponent bench. Shared across positions within the same role.

Both are added to the projected token vectors before the transformer encoder.

3.2.5 Transformer Encoder

The core of the policy network is a standard `nn.TransformerEncoder` with the following configuration:

Parameter	**Value**
Hidden dimension	512
Attention heads	8
Transformer layers	1
Feedforward expansion	×4 (2 048)
Activation	GELU
Dropout	0.1
Normalisation	Pre-norm (LayerNorm before attention/FFN)

The decision to use a single transformer layer was driven by an attention collapse analysis (see Section 3.4), which showed that deeper post-norm transformers suffered from gradient vanishing in layers 2–3, effectively reducing model depth to two functional layers.

Cross-Team Attention Bias

A learnable **attention bias** is added to the self-attention scores to encourage the model to attend to the opponent's active Pokémon, a critical source of tactical information:

From	To	Bias
CLS (0)	Opp. active (7)	+2.0
Our active (1)	Opp. active (7)	+2.0
Opp. active (7)	Our active (1)	+1.0
Opp. active (7)	Our bench (2–6)	+0.5
CLS (0)	Opp. bench (8–12)	+0.5

This bias was introduced after analysis showed that the default transformer progressively lost opponent attention in deeper layers. With a single layer and this bias, the model allocates 37–63 % of CLS attention to the opponent's active Pokémon.

3.2.6 Action Space

The agent operates in a **compressed action space** of 14 discrete actions:

Index	Category	Compressed	Native Mapping
0–3	Regular moves	Use move 1–4	Direct (or first legal variant)
4–7	Gimmick moves	Gimmick + move 1–4	Auto-selects: Mega → Z → Dynamax
8–13	Switches	Switch to slot 1–6	Mapped to party slot index

The native `poke-env` space has 22 actions (6 switches + 4 regular moves + 12 gimmick variants). Compression to 14 actions was motivated by:

- **Reduced branching factor:** 14 vs. 22 simplifies exploration.
- **Logical grouping:** Moves first, then gimmicks, then switches provides a consistent structure the model can learn.
- **Gimmick auto-selection:** Rather than requiring the agent to learn separate Mega/Z/Dynamax variants, the compressed space presents a single "gimmick" action that resolves to whichever gimmick is legally available.

Action Masking

At each step, the environment provides an `action_mask` of shape $(14,)$ indicating which actions are legal. The model applies a mask by setting invalid-action logits to -10^8 before the softmax.

This ensures the agent never selects an illegal action, regardless of its learned policy. The mask accounts for fainted Pokémon (cannot switch to them), unavailable moves (no PP, disabled), and the absence of gimmick conditions (e.g. Dynamax not available).

3.2.7 LSTM Memory

An optional **single-layer LSTM** (hidden size 512) can process the CLS token sequence across turns within a battle, providing cross-turn temporal memory:

- **Input:** CLS token embedding from the transformer at each turn.
- **State:** Hidden state h_t and cell state c_t are carried across turns, reset at episode boundaries.
- **Purpose:** Enable the agent to remember previous turns (e.g. which moves the opponent has used, whether Protect was used recently) rather than treating each turn independently.

The LSTM is enabled in the model code and processes both single-step and sequence batches. However, RLlib's connector path for recurrent state propagation required additional engineering (see Section 3.4.8).

3.2.8 Policy & Value Heads

Both heads share the transformer trunk (and LSTM, if active). The CLS token output serves as the shared representation:

- **Policy head:** hidden $\rightarrow$ Linear$(512, 256)$ $\rightarrow$ GELU $\rightarrow$ Linear$(256, 14)$. Output is action logits, masked before softmax.
- **Value head:** hidden $\rightarrow$ Linear$(512, 256)$ $\rightarrow$ GELU $\rightarrow$ Linear$(256, 1)$. Outputs scalar state-value estimate $V(s)$.

3.2.9 Model Size

The default (`standard`) configuration produces approximately **10–15 million parameters**. Preset variations are detailed in Section 3.3. Later `pure_league_play` runs used a smaller trunk (hidden dim 256, three transformer layers, LSTM disabled) while troubleshooting plateauing win rates; we did not see reliable gains from LSTM after the vocabulary/embedding migration, so recurrent memory was deprioritised in favour of stability.

3.3 Training Infrastructure

3.3.1 Distributed Training Architecture

Training is orchestrated by Ray RLlib, which distributes rollout collection across multiple workers and performs SGD updates on a central learner:

Component	Role
Main process	Configures training, manages curriculum callbacks, drives the training loop.
GPU learner	Performs PPO SGD updates on collected batches. Uses PyTorch backend.
Rollout workers	Each runs multiple environments, collects trajectories, and sends batches to the learner.
Showdown servers	8 local Node.js instances on ports 8000–8007, each hosting parallel battles.

The default (`standard`) configuration uses 24 workers with 8 environments per worker, yielding **192 parallel battles**. Each Showdown server hosts approximately 24 concurrent battles.

3.3.2 Hardware

Resource	Specification
GPU	NVIDIA RTX 5090 (`optimal` preset)
RAM	$\sim$44 GB peak
CPU	Multi-core (workers run on CPU threads)
Storage	Local SSD for checkpoints and MLflow artifacts

Memory management is critical: the project previously observed RAM growth from 8 GB to 52 GB over 5 hours due to stale entries in environment caches. This was mitigated with periodic cleanup and memory-leak monitoring.

3.3.3 Configuration Presets

Five hardware presets control model size, parallelism, and training duration:

Parameter	Quick	Standard	Memory Safe	Large
Hidden dim	128	512	256	768
Attention heads	8	8	4	12
Transformer layers	1	1	1	6
Embedding dim	32	32	32	32
LSTM hidden	—	512	256	768
Use LSTM	No	Yes	No	Yes
Train batch size	4 096	6 144	2 048	16 384
SGD minibatch size	256	512	128	512
Approx. params	∼1M	∼10M	∼4M	∼42M

The `quick` preset is used for smoke tests and debugging. The `standard` preset is the default for training runs. The `optimal` preset (not shown) matches `standard` but targets the RTX 5090 specifically. Production league runs use `pure_league_play` (Section 3.1.5) with different parallelism and PPO hyperparameters.

3.3.4 PPO Hyperparameters

Default hyperparameters across all presets (unless overridden):

Parameter	Value
Learning rate	2×10^{-4}
Discount (γ)	0.97
GAE lambda (λ)	0.87
Clip range (ϵ)	0.08
Entropy coefficient	0.013
Value-function loss coeff.	0.5
Value clip param	4.85
Gradient clipping	5.0
SGD iterations per batch	4

These are based on the first breakthrough run.

3.3.5 MLflow Experiment Tracking

All training runs are logged to an **MLflow** tracking server under the experiment name `Pokemon_RL_Battler`. The following categories of data are recorded:

- **Configuration snapshot:** Full training config, model config, curriculum config, and hardware preset, stored as MLflow parameters.
- **Training metrics:** Policy loss, value loss, entropy, KL divergence, win rate, episode length, per-stage statistics, logged every training iteration.
- **Validation metrics:** Win rates against fixed opponents (random, heuristic), per-opponent-type breakdowns, fallback event counts, logged at scheduled evaluation intervals.
- **Diagnostic metrics:** Action-mask density, observation sample recordings, HP remaining distributions, faint counts per episode.
- **System metrics:** CPU/RAM/GPU utilisation, memory leak counters.

Metrics use nested keys (e.g. `validation/fixed_paired/win_rate_vs_heuristic`) to support structured comparison in the MLflow UI.

3.3.6 Checkpoint Management

- **Frequency:** Checkpoints saved every 200 000 training steps.
- **Retention:** Up to 5 checkpoints retained (oldest automatically deleted).
- **Self-play export:** At each checkpoint, model weights are also exported to `checkpoints/selfplay_latest.p` for use by `SelfPlayPlayer` instances.
- **Resume:** Training can be resumed from the latest checkpoint using `--resume-checkpoint latest`, preserving MLflow run continuity via `--mlflow-run-id <RUN_ID>`.

3.3.7 Validation Pipeline

Scheduled evaluation runs against fixed opponent configurations:

Protocol	Description
Smoke test	Quick sanity check: 10 episodes against random opponent.
Fixed paired	20 frozen teams × 3 opponents (random + random no switch + heuristic) = 60 battles per evaluation run. Primary comparison metric.
Mirror	Both sides use the same team; tests raw battling skill without team advantage.

Validation is triggered at regular intervals (every 100 000 steps) during training and can also be run manually against any checkpoint via `scripts/validate_checkpoint.py`. Results are logged to MLflow with per-opponent-type breakdowns.

For `pure_league_play`, scheduled validation still uses the `benchmark` protocol (random-battle format) as a quick **out-of-distribution** generalisation check; future work is to add pool-matched evaluation aligned with training teams.

3.3.8 Training Throughput

Observed throughput was on the order of ∼600 environment steps per second on league-style runs (RTX 3090 training machine; a colleague's RTX 5090 did not remove the bottleneck). Showdown servers are CPU-bound (single-core Node.js instances), so GPU upgrades mainly help the learner, not battle simulation.

3.4 Iterative Development Methodology

The training pipeline was developed iteratively, with each phase addressing a specific problem identified through diagnostic analysis. The phases are presented in chronological order. Note that the curriculum configuration used during early development phases (baseline through PPO pipeline fixes) differed from the current four-stage system described in Section 3.1.4; the early curriculum used a simpler two-stage setup with "easy" (random opponents only) and "medium" (50/50 random and heuristic) stages.

3.4.1 Baseline Establishment

Starting point. The initial system used a 22-action native `poke-env` space, Adler-32 hash-based categorical embeddings (20 000 buckets), no positional encoding, a 4-layer post-norm transformer, and a disabled LSTM.

Approach. The training pipeline was run end-to-end with the early two-stage curriculum (easy: random opponents; medium: 50/50 random and heuristic opponents) to identify baseline behaviour and failure modes.

3.4.2 Validation Infrastructure

Problem. Without a standardised evaluation protocol, checkpoint quality could only be assessed by inspecting training-time metrics, which are noisy and may not reflect true policy strength.

Solution. Implemented a validation pipeline with multiple protocols:

Protocol	Method
Smoke	10 episodes vs. random opponent (quick sanity check).
Fixed paired	20 frozen teams $\times$ 2 opponents = 40 battles per run.
Mirror	Both sides use identical team.

Additionally, 20 frozen Gen 8 random-battle teams were generated and stored as JSON manifests (10 swapped pairs for fairness).

3.4.3 Vocabulary & ID Migration

Problem. Adler-32 hashing with modulo 20 000 produced an estimated 7.5 % collision rate among $\sim$1 800 unique entity strings. Collisions mean different Pokémon species (or items or abilities) share the same embedding, degrading the model's ability to distinguish between entities.

Solution. Replaced hash-based IDs with stable dictionary lookups:

- Generated explicit vocabularies from Showdown data files.
- Mapped each entity to a unique integer index.
- Added support for display-form aliases (e.g. "Shellos (East Sea)" $\rightarrow$ internal key).

Vocabulary sizes: 1 515 species, 583 items, 314 abilities. Coverage audit confirmed **zero unknown entities** across the BDSP dataset and validation manifests. This change intentionally broke compatibility with all previous checkpoints: new training runs are treated as a new experiment family.

3.4.4 Logging & Metrics Standardisation

Problem. Metric flow from environment to MLflow had gaps: "unknown" opponent metadata was not properly logged, and opponent-specific metrics were sometimes missing from training runs.

Solution. Traced the full metric path from environment through `EnvBridge` to MLflow. Fixed root causes:

- "Unknown" opponent metadata now correctly logged as a missing count.
- Opponent-specific keys (e.g. `training/win_rate_vs_random`, `training/win_rate_vs_heuristic`) now present in all runs.
- Defined a standard contract for high-signal metrics.

3.4.5 Embedding Audit & Global Features

Problem. The observation tensor lacked high-value global context features that could help the model understand battle dynamics (e.g. which opponent type it was facing, whether it was forced to switch).

Solution. Added to the global token (position 0) without changing the tensor dimensions:

- Opponent label categories (`opponent_random`, `opponent_heuristic`, `opponent_other`).
- Normalised battle turn, force-switch flag, trapped flag.
- Normalised available move and switch counts.
- Dynamax, Mega Evolution, and Z-Move availability flags.

Also discovered and fixed **broken weather encoding**: the battle state passed weather as a Python dictionary rather than enum keys, so weather information was never actually encoded into the observation.

3.4.6 Position & Role Embeddings

Problem. The transformer received 13 tokens with no positional information. Self-attention is permutation-invariant, meaning the model could not structurally distinguish between the active Pokémon and bench slots, or between own-team and opponent-team tokens.

Solution. Added two sets of learnable embeddings after input projection:

- **Position embeddings:** 13 vectors, one per token position.
- **Role embeddings:** 5 coarse roles (CLS, our active, our bench, opponent active, opponent bench), shared across positions within a role.

Old checkpoints are incompatible due to new embedding parameters, so a fresh training run was required.

3.4.7 Compressed Action Space

Problem. The native `poke-env` action space has 22 actions (6 switches + 4 regular moves + 12 gimmick variants). This large branching factor slows exploration, and the 12 gimmick variants are rarely all available simultaneously.

Solution. Introduced a compressed 14-action space:

Indices	Category	Mapping
0–3	Regular moves	Move 1–4
4–7	Gimmick moves	Auto: Mega → Z → Dynamax
8–13	Switches	Party slot 1–6

Gimmick slots automatically resolve to the first legally available gimmick type, so the agent only needs to learn "use gimmick" rather than "use specific gimmick variant." Action mask changed from $(22,)$ to $(14,)$. Incompatible with old checkpoints.

3.4.8 LSTM Memory Fix

Problem. The LSTM path existed in model code but was non-functional due to RLlib's connector path not supporting recurrent state propagation in the production setup.

Solution. Workaround LSTM integration:

- LSTM now runs even when no incoming recurrent state is provided (creates zero initial state).
- `compute_values()` uses the same forward path as policy logit computation, ensuring consistent state handling.
- Single-step and sequence batches both pass through the recurrent path correctly.
- `get_initial_state()` returns unbatched state tensors of shape $[H]$ for compatibility with RLlib.

End-to-end training validation with LSTM enabled remains a next step.

3.4.9 PPO Pipeline Corrections

Problem. After the architectural improvements, a comprehensive audit revealed that the PPO training pipeline itself had six issues preventing effective learning, independent of model architecture. **Six fixes applied:**

Issue	Before → After	Impact
Gradient clipping starvation	clip = 0.5 → 5.0	Total grad norm ∼65 was clipped to 0.77 %, effectively zeroing all transformer/embedding gradients.
No per-step action signal	0.0 → 0.2/0.3	Matchup and action-quality weights restored, providing per-step credit assignment (without them, gradients cancelled over episodes).
Entropy collapse	0.005 → 0.2	Entropy bonus was negligible (∼0.02 vs. ∼5.0 total loss). Agent stopped exploring.
Discount horizon too long	$\gamma = 0.99 \rightarrow 0.95$	100-step horizon for ∼25-turn battles was too long. 20-step horizon better matches episode length.
Value function instability	vf_clip = 25.0 → 10.0	Allowed 3× the reward range per update, destabilising value regression.
LSTM crash on missing state	ValueError → zeros	`_extract_state` crashed when state was absent instead of returning a zero tensor.

Key Insight

Before diagnosing model architecture, it is essential to verify that gradients are actually flowing. In this case, an aggressive gradient clip was silently starving all learnable layers, and the problem was invisible in standard loss metrics.

3.4.10 Reward Scaling Approach

Problem. Across the entire project history, explained variance had been stuck at approximately 0.1 (and sometimes negative). The value function was essentially predicting a constant, and value loss showed no downward trend. This was the single largest blocker to policy improvement.

Root cause. Returns were in the range $[-15, +15]$ with no normalisation. RLlib normalises advantages but *not* returns, so the value function was chasing an unscaled, moving target. With `vf_clip_param`=10.0, the value head could overshoot by 3× the reward range per update, preventing convergence.

Changes applied:

Parameter	Before	After	Rationale
`reward_scale`	—	0.1	Scales returns from $[-15, +15]$ to $[-1.5, +1.5]$. Makes value regression tractable.
`vf_clip_param`	10.0	3.0	Tighter clipping prevents value head from overshooting.
λ (GAE)	0.92	0.88	Reduces GAE target variance.
`hidden_dim`	256	512	Wider model gives value head more capacity.
`num_heads`	4	8	More heads for finer-grained attention.

3.4.11 Fixed Team & No-Gimmick Format

Problem. Random battles introduce high entropy in team composition, making it difficult to isolate whether the agent is learning battle strategy or merely memorising matchup patterns. Additionally, Dynamax and Terastallize mechanics add complexity that distracts from core battle competency.

Solution. Two changes:

- **Fixed player team:** Added `player_team_path` config option. When set, the agent always uses a specific team (Showdown `.txt` format). The battle format automatically switches from `gen8randombattle` to `gen8customgame`.
- **No-gimmick formats:** Created custom Showdown formats (`gen8customgamenogimmicks`, `gen8randombattlenogimmicks`) that disable Dynamax, and remove Sleep Clause for more natural battle dynamics.

With a fixed team, the agent only needs to learn battle strategy, not team adaptation. This simplified the learning problem and provided a controlled setting for validating the training pipeline.

3.4.12 Self-Play Infrastructure

Problem. Self-play opponents were not actually using the training model's weights, playing with random initialisation instead.

Root causes:

1. **Weights never loaded:** The `selfplay_weights_path` was relative (`checkpoints/selfplay_latest.pt`). Ray workers run from a temporary directory and could not find the file. The load failure was silently swallowed by a bare `except Exception: pass`.
2. **Stale weights:** Weights were only exported every 150K steps. The opponent played with outdated checkpoints.
3. **Deterministic opponent:** Argmax action selection allowed the training agent to exploit deterministic patterns.

Fixes:

- Resolved `selfplay_weights_path` to absolute before passing to Ray workers.
- Replaced `except: pass` with explicit error logging.
- Weight export now happens every training iteration (not just at checkpoint saves).
- Replaced argmax with temperature sampling ($\tau = 0.8$).
- Fixed `opponent_type=None` to `opponent_type="self"`, ensuring correct embedding features.

After the fix, all workers successfully loaded weights (previously zero workers).

Key Insight

Any file path passed to Ray workers must be resolved to an absolute path *before* serialisation. Ray workers execute from a temporary working directory and cannot find relative paths. Silent exception handling can mask critical failures for thousands of training steps.

3.4.13 NaN Bug Workaround

Problem. After fixing weight loading, the self-play opponent's fallback rate was 100 %, meaning it still played randomly.

Root cause. PyTorch 2.10 has a bug in `TransformerEncoderLayer.forward()` when using a float-valued `src_mask`. The model's learnable attention bias (an `nn.Parameter`) triggered this code path, producing NaN logits every forward pass. The `multinomial` sampler then failed, falling back to random action selection.

Fix. Replaced `layer(x, src_mask=bias)` with a manual norm-first decomposition: LayerNorm → self-attention → residual → LayerNorm → FFN → residual. This bypasses PyTorch's fused path and produces correct values.

Diagnostic instrumentation added:

- `_diag` accumulator in `SelfPlayPlayer`: weight load count, fallback count, action histogram, top probability, entropy, valid action count.
- Threaded through wrapper → env bridge → trainer, logged to `logs/selfplay_diagnostics.log` and MLflow.

Key Insight

Do not trust framework-level operations without testing. A PyTorch internal bug in the transformer layer was causing NaN outputs, and only explicit diagnostic logging revealed it. This was discovered because self-play win rate was basically the same as against random, while it should have been near 50 %.

3.4.14 Expanded Validation System

Problem. The validation system had several limitations: subprocess per protocol, only argmax actions, missing `random_no_switch` opponent, no unified benchmark, and no console output during training. This made systematic hyperparameter optimisation difficult.

Solution. Major overhaul of the validation infrastructure:

- **Benchmark protocol:** 3 opponents × 50 episodes (150 total). Reports per-opponent win rate with Wilson 95 % confidence intervals, a weighted skill score (random=1.0, `random_no_switch`=1.5, heuristic=2.0), and consistency (fraction of tiers above 50 % win rate).
- **In-process validation:** `run_inprocess_validation()` runs against the live algorithm during training, with no subprocess, Ray init, or checkpoint restore overhead.
- **Parallel benchmark:** Episodes distributed across N servers via `ThreadPoolExecutor`. Each environment gets a sequential chunk, environments run in parallel.
- **Stochastic evaluation:** `--explore` flag samples from a masked softmax instead of argmax, evaluating the stochastic policy.
- **Console output:** Per-opponent win rate bars with confidence interval bounds printed at each validation interval.

3.4.15 Remote Training and CUDA Stability

Long league runs initially hit recurring `CUDA illegal memory access` errors on a local workstation. We tried driver reinstalls, memory tuning, and smaller batch sizes; the failures persisted and correlated with OS-level instability on that machine. **Disclaimer:** AI-assisted debugging was used to narrow hypotheses, but the practical fix was moving training to a **vast.ai** cloud instance with a clean GPU stack, where the same code completed a full 25M-step run without those crashes.

3.4.16 Hyperparameter Sweep Methodology

Problem. With fixed teams performing well, the remaining performance gap (particularly against heuristic opponents with random teams) required systematic hyperparameter optimisation rather than manual tuning.

Solution. Implemented an Optuna-based sweep with TPE sampler:

- **Objective:** Benchmark skill score (weighted win rate across three opponent tiers).
- **Fixed curriculum:** Single stage with 20 % random, 40 % `random_no_switch`, 40 % heuristic. No stage promotion, ensuring consistent evaluation across trials.
- **Search space:** 10 parameters covering learning rate (log-uniform), entropy coefficient (log-uniform), discount factor, GAE λ, PPO clip, value-function clip, dropout, reward scale (categorical), matchup weight, and action-quality weight.
- **Resume mechanism:** SQLite persistence (`logs/hparam_study.db`) allows interrupting and resuming sweeps.

4. Results & Analysis

This chapter presents the experimental outcomes of the training pipeline. Results are reported for two distinct settings: **fixed teams** (Milestone 1) and **random teams** (Milestone 2). We first define the evaluation metric, then trace the development progression chronologically, and finally assess each milestone independently.

4.1 Benchmark Metric

All validation results use the **benchmark protocol** (3 opponents × 50 episodes = 150 total). The primary metric is the **skill score**, a weighted average of per-opponent win rates:

$$\text{skill\_score} = \frac{1.0 \times \text{WR}_{\text{random}} + 1.5 \times \text{WR}_{\text{random\_no\_switch}} + 2.0 \times \text{WR}_{\text{heuristic}}}{1.0 + 1.5 + 2.0}$$

Skill score ranges from 0 (loses all) to 1 (wins all). The weighting reflects opponent difficulty: beating the heuristic opponent contributes twice as much as beating the random opponent. Per-opponent win rates are reported with Wilson 95 % confidence intervals.

4.2 Development Progression

This section traces the experimental outcomes chronologically, corresponding to the development phases described in Section 3.4. Note that the early phases used a different curriculum configuration from the current four-stage system (Section 3.1.4): the early setup had two stages, "easy" (random opponents only) and "medium" (50/50 random and heuristic).

4.2.1 Baseline Results

With the initial system (22-action space, hash-based embeddings, 4-layer post-norm transformer, no positional encoding), the training pipeline ran end-to-end but produced limited learning:

- **Easy stage** (random opponents only): promoted at ∼84K steps with 85 % win rate.
- **Medium stage** (50/50 random and heuristic): plateaued at ∼50 % win rate by ∼100K steps and did not improve further over 2.5M+ steps. This aggregate win rate masked a stark split: 100 % win rate against random opponents and 0 % against heuristic opponents.
- **Loss signature:** High value loss, high entropy, low policy loss, indicating the value network was failing to predict returns while the policy was not making meaningful updates.

4.2.2 Validation Infrastructure Results

First validation results (checkpoint at step 51 200):

- Overall win rate: 37.5 %
- Vs. random: 75 %
- Vs. heuristic: 0 %
- Fallback events: 2.60 per battle (0.069 per step), indicating frequent invalid-action recoveries.

A subsequent diagnostics run with the final checkpoint showed:

- Overall: 50 %
- Vs. random: 95 %

- Vs. heuristic: 5 %

The sharp contrast between random and heuristic performance confirmed that the agent had learned basic move selection but not strategic play.

4.2.3 Observation Pipeline Improvement Effects

After the vocabulary migration, embedding audit, global features, and positional/role embeddings were added, the agent's win rate against random opponents improved from 65 % to 85 %, confirming that the observation pipeline bugs had been degrading performance.

4.2.4 PPO Pipeline Fix Outcomes

The six PPO pipeline corrections collectively enabled the agent to start learning beyond the easy curriculum stage. The most impactful single change was gradient clipping: with the clip set to 0.5, the transformer and embedding layers received near-zero gradients for the entire training run.

4.2.5 Reward Scaling Results

The reward scaling fix was the turning point for training performance. Measured over 378K steps:

- Explained variance: $\sim$0.1 $\rightarrow$ **0.53**–**0.87** (median $\sim$0.72).
- Value loss: converged from 0.21 to **0.08**–**0.13** by step 15K.
- Benchmark at 200K steps (fixed team): **100 %** vs. random, **96 %** vs. `random_no_switch`, 22 % vs. heuristic.

4.2.6 Self-Play Fix Results

After the self-play weight-loading fix, all 12 workers successfully loaded weights (previously zero workers). Self-play then produced meaningful training signal.

4.2.7 NaN Fix Results

After the NaN workaround, the self-play fallback rate dropped from 100 % to 0 %. Over 500K steps of training (with randomly generated teams on both sides), the model's confidence progressed from near-uniform (entropy 2.0, top probability 0.21) to confident (entropy 1.2, top probability 0.47):

Metric	Value	Interpretation
Fallback rate	0 %	Self-play fully functional
Mapping fallback	0 %	Action resolution correct
Top probability	0.21 $\rightarrow$ 0.47	Policy becoming confident
Entropy	2.0 $\rightarrow$ 1.2	Converging from uniform
Win rate vs. random	91 %	Strong baseline competency
Win rate vs. `random_no_switch`	31 %	Room for improvement
Win rate vs. self	30 %	(10 games only, high variance)

4.2.8 Fixed Team Results

With a fixed player team and no gimmicks, the agent achieved near-perfect performance, demonstrating that the architecture, observation pipeline, and reward design are sufficient when team variability is removed. Detailed results are in Section 4.3.

4.2.9 Hyperparameter Sweep Results

Dry run (3 trials, 100K steps each, fixed team): Best trial achieved 70 % win rate vs. heuristic at 150K steps (skill score 0.847), confirming the sweep can find productive regions of the hyperparameter space.

Full sweep (50 trials, 600K steps each, random teams): None of the configurations reached even a 40 % win rate against heuristics. Fixed-team trials saturated the benchmark within 300K steps with most configurations in the 96 % to 98 % win rate range against heuristics, with differences largely attributable to noise.

4.3 Fixed Teams (Milestone 1)

With a fixed player team and no gimmicks, the agent achieves strong performance:

Metric	Value
Benchmark skill score	0.9911
Win rate vs. random	100 %
Win rate vs. `random_no_switch`	96 %+
Win rate vs. heuristic	98 %

This result demonstrates that the agent can learn a highly effective battle strategy when the team is known and consistent. The near-perfect performance against all three opponent tiers indicates that the architecture, observation pipeline, and reward design are sufficient for fixed-team play.

4.3.1 Value Function Improvement

The reward scaling fix (Section 3.4.10) was the turning point for the fixed-team setting:

Metric	Before Scaling	After Scaling
Explained variance	∼0.1 (sometimes negative)	0.53–0.87 (median 0.72)
Value loss	High, no trend	0.08–0.13 (converged by 15K)
Win rate vs. heuristic	∼5 %	98 %

4.3.2 Training Curve

4.4 Team-Pool League Training (New)

A later training line (`pure_league_play`, 25M steps, team pools on `gen8customgame`) used the curriculum in Section 3.1.5. We compared two variants: (1) a fixed 20-team pool from the start (13.7M steps), and (2) a staged pool curriculum (3 → 5 → 10 → 20 teams; 25M steps). Table 4.1 reports the final logged snapshot values for in-distribution win rates in the league environment (note: single snapshot, not a trailing-window average).

Metric (in-distribution)	20-team (13.7M)	3–5–10–20 (25M)
Win rate vs. heuristic	28.7 %	78.3 %
Win rate vs. `random_no_switch`	88.9 %	81.3 %
Win rate vs. random	100 %	100 %

Table 4.1: Snapshot comparison of league outcome win rates for a fixed 20-team pool vs. staged pool curriculum.

Scheduled `benchmark` validation (random-battle format, different from training) remained much lower against heuristics; we keep it as a coarse out-of-distribution probe and plan to add pool-matched evaluation in future iterations.

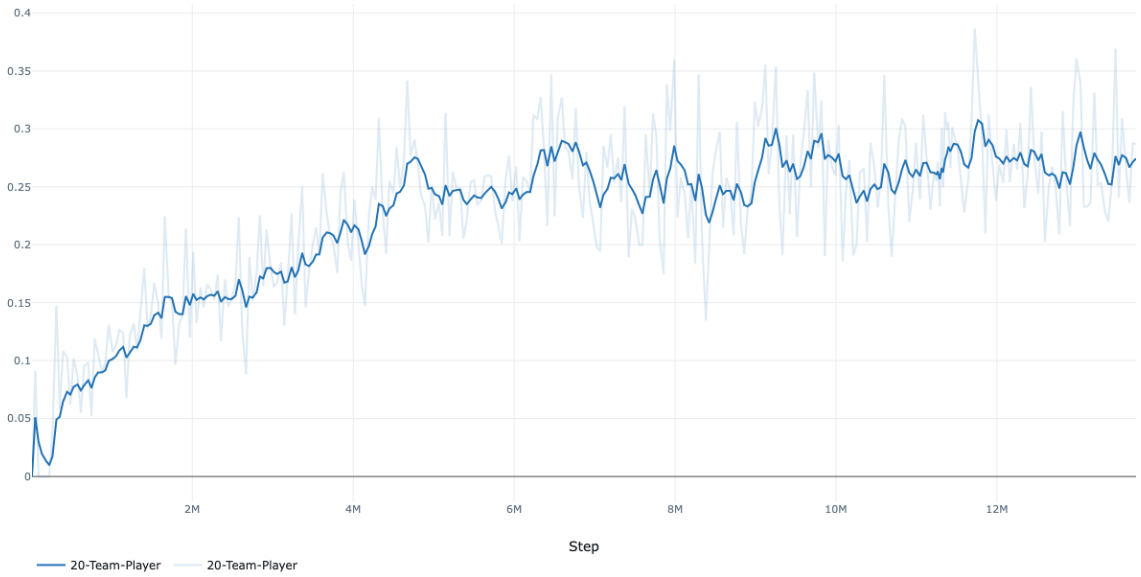

Figure 4.1: Fixed 20-team pool: outcome win rate vs. heuristic over training steps (plateau around 30 %).

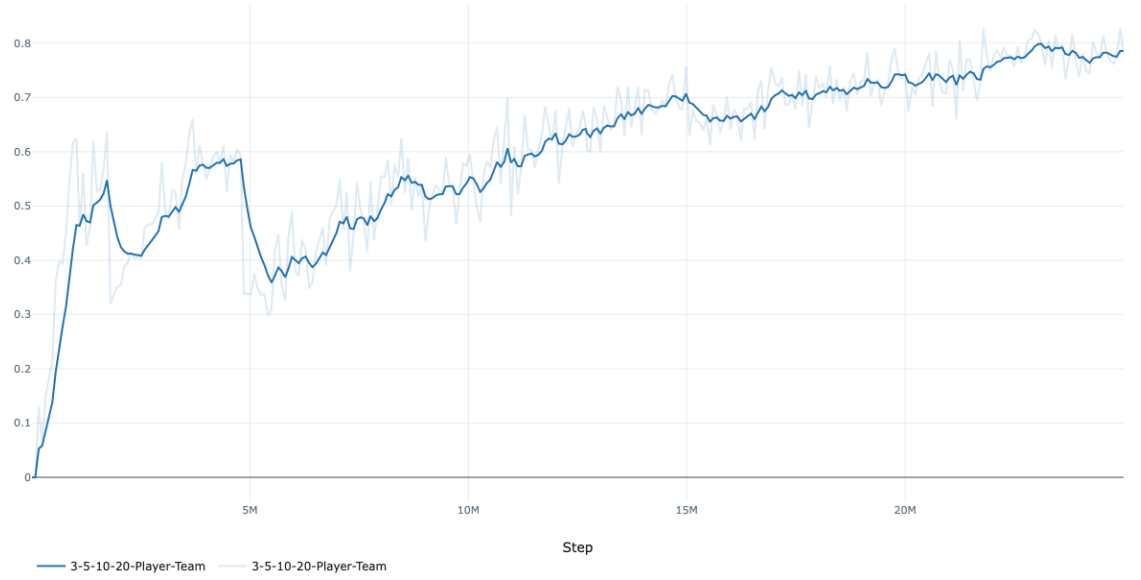

Figure 4.2: Staged pool curriculum ($3 \to 5 \to 10 \to 20$ teams): outcome win rate vs. heuristic over training steps (climbing to $\sim$80 %).

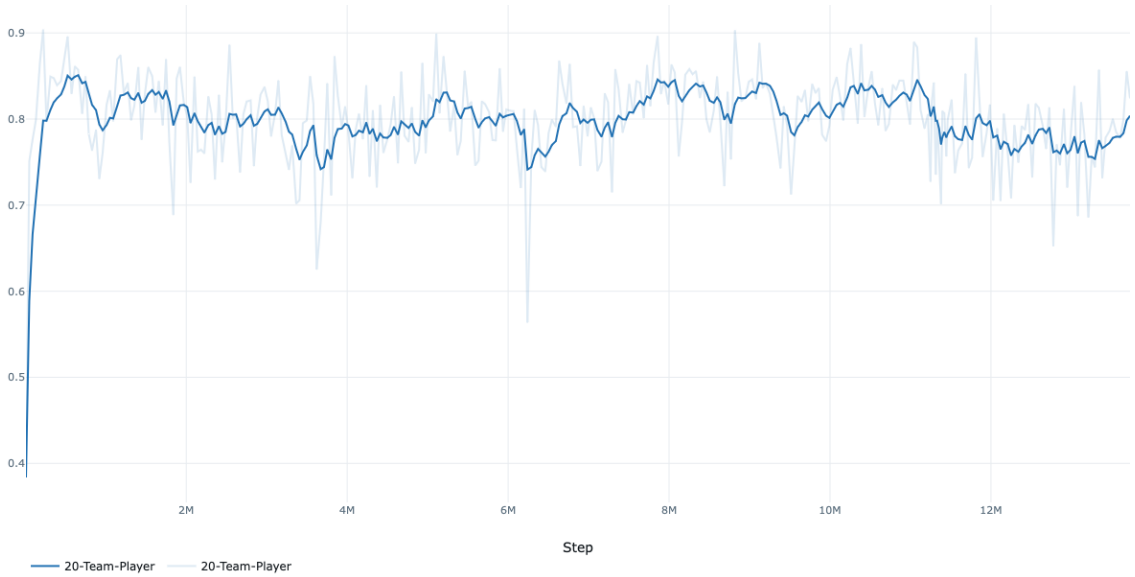

Figure 4.3: Fixed 20-team pool: PPO explained variance over training steps (exported from MLflow).

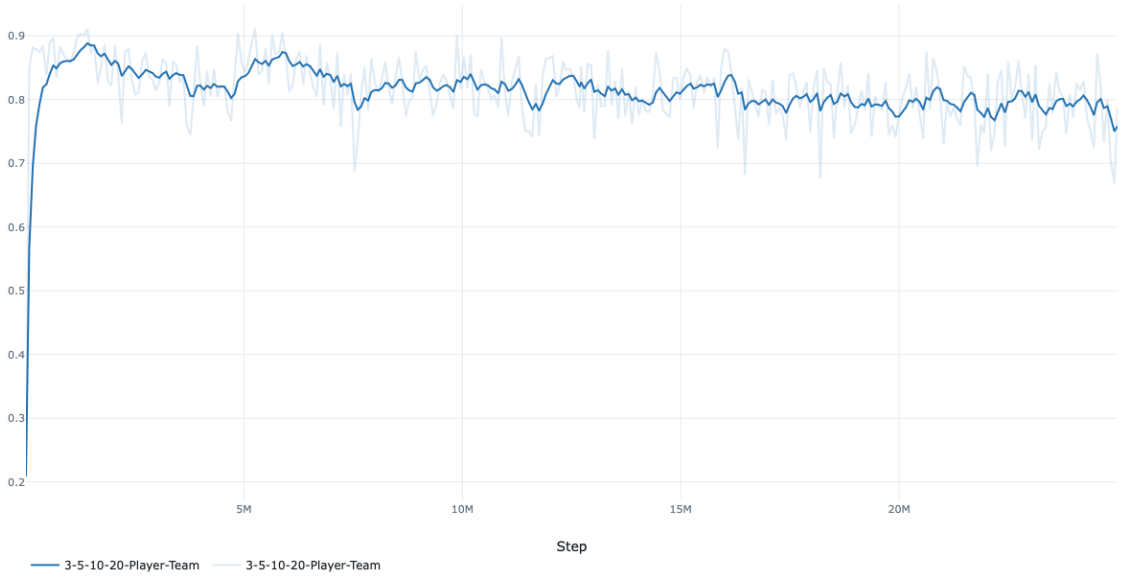

Figure 4.4: Staged pool curriculum ($3 \rightarrow 5 \rightarrow 10 \rightarrow 20$ teams): PPO explained variance over training steps (exported from MLflow).

A checkpoint export for the 25M run is in `saved_models/pure_league_play_25M/`.

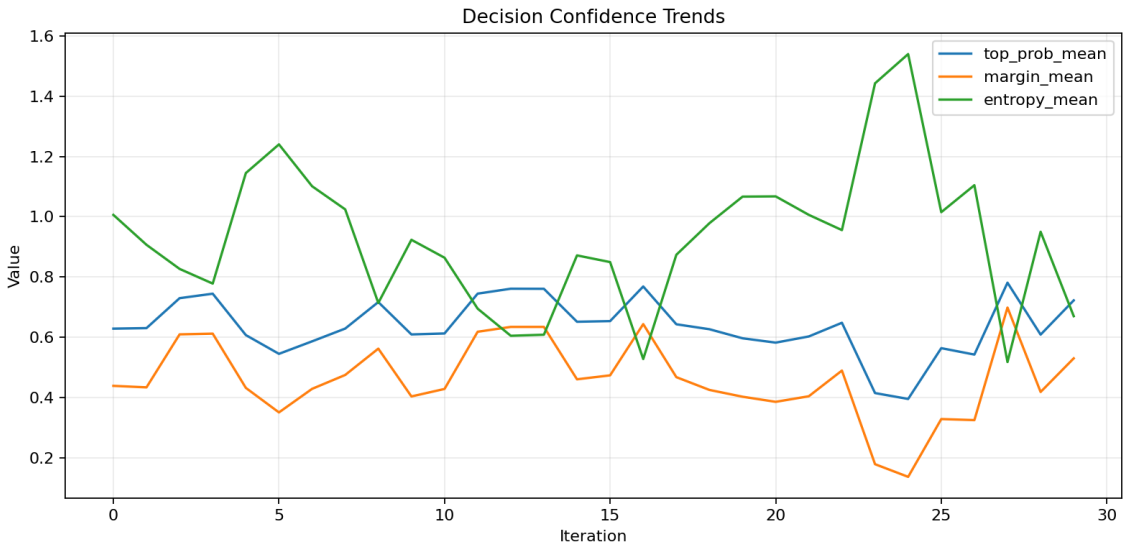

Figure 4.5: Decision-diagnostics confidence trends from the final checkpoint (sampled battles; not a training-time metric).

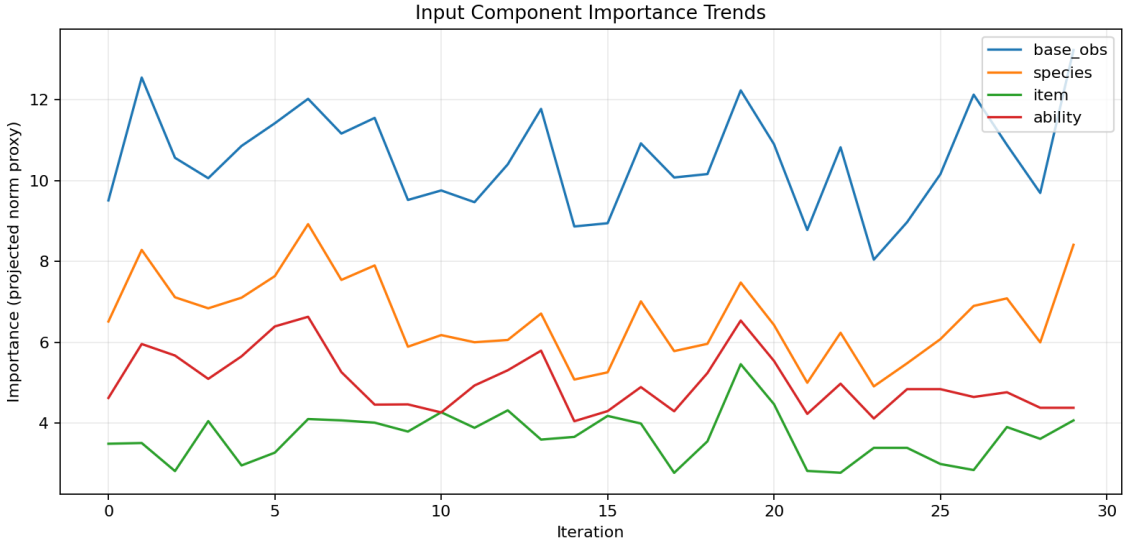

Figure 4.6: Decision-diagnostics input component importance trends from the final checkpoint.

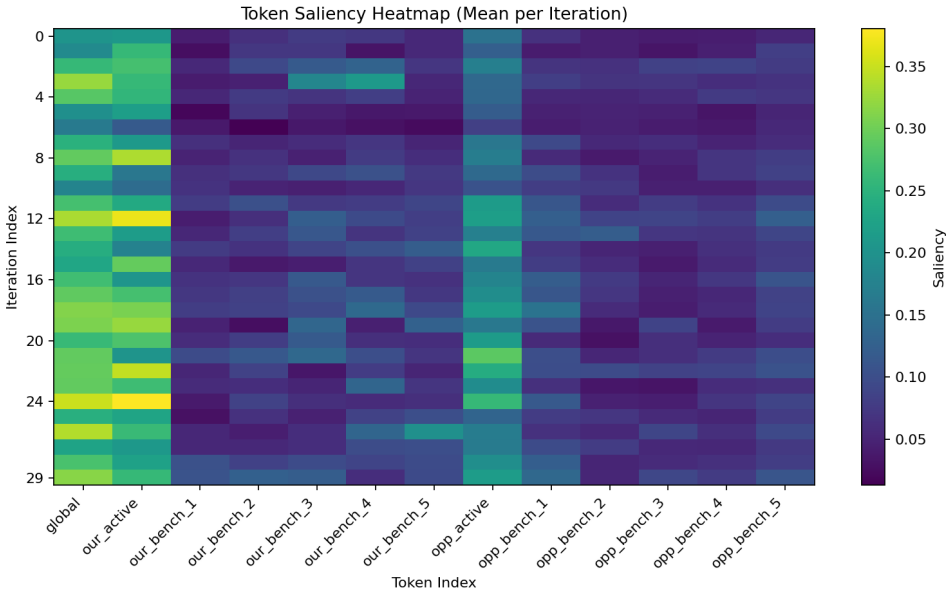

Figure 4.7: Decision-diagnostics token saliency heatmap (final checkpoint).

4.5 Random Teams (Milestone 2)

With randomly generated teams for both sides (the standard `gen8randombattle` format), the problem is substantially harder. Each battle involves unseen teams on both sides (the agent can see its own team), requiring the agent to generalise rather than memorise a single team's strategy.

4.5.1 Current Best Results

Metric	Value
Benchmark skill score (best trial)	0.3733
Win rate vs. random	84%
Win rate vs. `random_no_switch`	48%
Win rate vs. heuristic	8 %

The skill score of 0.3733 reflects the difficulty of generalisation: the agent can beat random opponents reliably but struggles against the heuristic. This is expected given the high entropy in team composition. However the performance against random_no_switch is still very disappointing.

4.5.2 Hyperparameter Sweep Status

The Optuna sweep was made with the following setup:

- 50 trials planned, 600K steps per trial.
- Single fixed curriculum stage (no promotion), ensuring comparable evaluation across trials.
- Objective: benchmark skill score.

Results were lackluster as there were no major brekthrough findings or large divergences in capability with random teams.

4.5.3 Noise in Benchmark Scores

An important observation is that the benchmark skill score has significant variance due to the stochastic nature of random-team battles. Lucky team matchups can inflate the score above what the peak training win rates would imply. With only 50 episodes per opponent tier, the Wilson confidence intervals are wide, particularly for the heuristic tier where win rates are low. Future work should increase episode counts for more stable evaluation.

4.6 Key Insight: Reward Scaling as the Catalyst

The central finding from the experimental programme is that **reward scaling was the single most impactful change**:

Key Insight

For the entire project history, the value function was non-functional (explained variance ∼0.1) despite multiple architectural improvements. A single change, adding `reward_scale` = 0.1 to normalise returns from $[-15, +15]$ to $[-1.5, +1.5]$, caused explained variance to jump to 0.53–0.87 and unlocked competency against heuristic opponents (fixed teams). Combined with fixed teams, this produced a benchmark skill score of 0.9911 with 98 % win rate vs. heuristic.

The lesson: **normalise the return scale before tuning anything else.** The value function must be able to learn before the policy can improve.

This redirects the optimisation focus for the random-teams setting from architecture to training dynamics: the same reward-scaling insight applies, but the higher entropy of random teams requires additional work on generalisation. This problem remains unsolved for now.

4.7 Honest Assessment

At the time of writing, the project demonstrates:

What works	What still needs work
Fixed teams: near-perfect performance (skill score 0.99)	Random teams: 0.37 skill score, 8 % vs. heuristic
Value function learns after reward scaling (EV 0.72)	Benchmark score has high variance at 50 eps/tier
Self-play infrastructure functional (0 % fallback)	LSTM end-to-end validation showed no improvement in performance
Distributed training stable at 48 environments	Gauntlet validation pipeline not yet automated
MLflow tracking produces complete, comparable metrics	Complete information random vs random as a test
Curriculum learning provides structured progression	Team builder (Milestone 3) not yet started

The engineering foundation is solid and Milestone 1 (fixed teams) is essentially solved. The remaining work focuses on Milestone 2 (random teams) and the team builder for Milestone 3 is out of scope.

5. Risk, Ethics & Considerations

5.1 Risk Assessment

5.1.1 Technical Risks

Risk	Impact	Mitigation
Medium-stage plateau	High	Systematic debugging through controlled experiments (positional encoding, embeddings, reward shaping). Multiple architectural improvements have already been validated.
Memory leaks	Medium	Observed RAM growth from 8 GB to 52 GB over 5 hours. Mitigated with periodic cleanup, memory-leak counters, and the `memory_safe` preset for limited hardware.
RLlib API instability	Medium	The LSTM integration required workarounds for RLlib's connector path. Custom `RLModule` and `EnvRunner` implementations provide a stable interface.
Reward hacking	Low	Multi-component reward with matchup and action-quality signals reduces the risk of the agent exploiting a single reward channel.

5.1.2 Timeline Risks

The critical path depends on resolving the medium-stage plateau. If the reward signal redesign does not produce improvement within 2–3 experimental iterations, the timeline for Milestone 2 completion could extend by an additional 2–4 weeks. This means that there is high likely hood of not being able to beat Milestone 2.

5.2 Ethical Considerations

5.2.1 Domain Context

This project operates entirely within the domain of **game AI**. The agent plays Pokémon battles against simulated opponents in a controlled environment. There are no ethical concerns related to:

- **Personal data:** No personal or sensitive data is collected, stored, or processed. All data consists of game mechanics, trainer rosters, and battle states.
- **Harmful applications:** The trained model cannot be repurposed for harmful real-world applications. It is specialised for a turn-based game with a fixed action space.
- **Bias or fairness:** The opponent populations are programmatically defined. There are no human subjects, user-facing decisions, or societal impacts.

5.2.2 Software Licences

Component	Licence
Pokémon Showdown	AGPL-3.0 (open source, self-hosted)
`poke-env`	MIT
Ray / RLlib	Apache-2.0
PyTorch	BSD-3-Clause
MLflow	Apache-2.0

All dependencies are open-source. The project does not redistribute copyrighted Pokémon assets; it uses the Showdown simulator's built-in data under fair use for research and educational purposes.

5.2.3 EU AI Act Classification

Under the EU AI Act, this system falls into the **minimal risk** category. It is a research prototype for a game environment with no interaction with natural persons, no biometric data, no critical infrastructure, and no social scoring. No conformity assessment is required.

5.3 Human-in-the-Loop Considerations

Although the training loop is automated, several critical decisions require human judgement:

- **Checkpoint selection:** A human reviews MLflow dashboards to select the best checkpoint for gauntlet evaluation, rather than automatically selecting by training win rate (which can be misleading).
- **New feature selection:** A human reviews the MLflow dashboards and other diagnostics to prioritize the next feature to implement.
- **Curriculum stage definition:** If a training run stagnates, a human may manually adjust the curriculum stage, opponent mix, or promotion thresholds. There is no automated parameter tuning for this stage.
- **Hyperparameter tuning:** Learning rate, entropy coefficient, and reward weights are set based on human interpretation of training curves, not automated tuning.
- **Experiment design:** The sequence of architectural changes (positional encoding, embeddings, action compression) was guided by human analysis of failure modes, not random search.

This human oversight is essential because the environment is complex enough that automated optimisation could converge to local optima that do not generalise to the gauntlet.

5.4 Limitations

- The agent has been tested only on Milestone 1 (fixed team vs. random opponents). And is currently failing on Milestone 2. Completing Milestone 2 and Team builder integration (Milestone 3) and full gauntlet completion (Milestone 4) remain future work.
- The system operates in the Gen 8 random-battle format. Transfer to other Pokémon formats would require observation-space adjustments.
- All training and evaluation use local hardware (no cloud deployment).
- The heuristic opponent (`SimpleHeuristicsPlayer`) is a rule-based baseline, not a strong competitive AI. Success against it does not guarantee competence against human players or more sophisticated agents.

6. Conclusions & Next Steps

6.1 Summary of Achievements

Over the course of this project, we have built a complete, reproducible RL training system for Pokémon battles:

- **End-to-end training pipeline:** From Showdown server management through distributed PPO training to MLflow-tracked evaluation, the system requires only a single command to start a training run.
- **Distributed infrastructure:** 48 parallel environments across 8 Showdown servers, coordinated by Ray RLlib with custom environment connectors.
- **Transformer-based policy:** A 10M-parameter model with vocabulary embeddings, positional and role encodings, cross-team attention bias, and compressed 14-action space.
- **Systematic debugging:** Three observation pipeline bugs found and fixed (weather encoding, switch alignment, mask contamination). Attention collapse in deep transformers diagnosed and resolved. Self-play weight loading failure identified through diagnostic logging.
- **Reward scaling breakthrough:** The single most impactful change in the project. Normalising returns `reward_scale` unlocked value-function learning (explained variance $\sim$0.1 $\rightarrow$ 0.72) and enabled strong performance against heuristic opponents.
- **Milestone 1 essentially solved:** With fixed teams, the agent achieves a benchmark skill score of 0.9911 and 98 % win rate against heuristic opponents.
- **Validation suite:** Benchmark protocol with per-opponent win rates, Wilson confidence intervals, skill score, and consistency metrics. In-process evaluation during training with zero overhead.
- **Self-play infrastructure:** Hot-reloading opponent with temperature sampling, NaN-safe transformer forward pass, and comprehensive diagnostic logging.
- **Hyperparameter sweep:** Optuna-based automated tuning with SQLite persistence, currently running for the random-teams setting.

6.2 Lessons Learned

1. **Normalise the return scale first.** The value function must be able to learn before the policy can improve. In this project, reward scaling ($\times$0.1) was more impactful than any architectural change. Explained variance jumped from $\sim$0.1 to 0.72, and win rate vs. heuristic went from $\sim$5 % to 98 % (fixed teams).
2. **Systematic debugging is essential.** Three bugs in the observation pipeline collectively affected 14 % of training steps. A critical weight-loading bug in self-play was hidden by silent exception handling. Without diagnostic logging and pipeline validators, these would have been nearly impossible to identify.
3. **Attention collapse is real.** In 4-layer post-norm transformers, deeper layers become dead weight. Measuring attention distributions, not just loss curves, is necessary to diagnose whether the model is actually using its capacity.
4. **Reward design outweighs architecture tuning.** The biggest performance gains came from fixing observation bugs (win rate vs. random: 65 % $\rightarrow$ 85 %) and reward scaling (win rate vs. heuristic: 5 % $\rightarrow$ 98 %), not from architectural changes. Once the architecture was no

longer the bottleneck, further architecture changes produced no win-rate improvement.

5. **Curriculum learning provides structure but can mask problems.** The easy stage was solved quickly, giving a false sense of progress. The real challenge only appeared when facing heuristic opponents.
6. **Metrics must be tracked by opponent type.** An aggregate "50 % win rate" obscured the fact that the agent was winning all random games and losing all heuristic games.
7. **File paths in distributed systems must be absolute.** Ray workers execute from a temporary directory and cannot resolve relative paths. This caused a critical self-play failure that was hidden by exception swallowing.
8. **Framework bugs happen.** A PyTorch 2.10 bug in transformer attention with float masks caused NaN outputs. Manual decomposition was required. Trusting framework internals without testing can waste time.

6.3 What Did Not Work

- **4-layer post-norm transformer:** Layers 2–3 were dead (uniform attention). Effective depth: 2 out of 4 layers.
- **Hash-based embeddings:** 7.5 % collision rate degraded entity distinction.
- **LSTM:** Was a lot of work but in no way could we make sure if it makes any difference. Random teams failed regardless and fixed temas won regardless of LSTM. As such it just took up training time.
- **Gradient clipping at 0.5:** Silently starved all transformer and embedding layers. The problem was invisible in loss metrics.
- **Training without reward scaling:** The value function could not learn from unscaled returns in $[-15, +15]$, resulting in negative explained variance for the entire project history up to the fix.

6.4 Actionable Next Steps

1. **Feature Engineering:** Think about new and good features to engineer to reduce the amount of learning an agent has to learn at a time. For example type match-ups or damage % predictions based on previous turns.
2. **Complete information random-teams hyperparameter sweep:** Implementing a random vs random team format in which the agent gets complete opponent team information.
3. **Improve random-teams performance:** Sweep does not reach satisfactory performance, investigate auxiliary tasks (opponent-move prediction, state-value prediction as a supervised objective) and more granular shaped rewards.
4. **Increase benchmark stability:** Raise episodes per opponent tier from 50 to 100+ to reduce variance in the skill score.
5. **Automate gauntlet validation:** Implement evaluation against the full BDSP gauntlet sequence with fixed seeds, deterministic configuration, and standardised metric outputs.
6. **Begin Milestone 3 (team building):** Investigate approaches for a team-composition agent, likely requiring a separate model or a hierarchical policy structure.

6.5 Milestone Outlook

Milestone	Status	Path Forward
M1: Fixed Teams	**Completed**	Benchmark skill score 0.9911, 98 % vs. heuristic. Formal gauntlet validation pending.
M2: Random Teams	In progress	Best skill score 0.3733. Target: competitive with heuristic opponents.
Team-pool league	New	`pure_league_play`: strong in-distribution WR vs. heuristic (∼80 %); OOD benchmark still weak. 25M checkpoint saved.
M3: Team Building	Future	Requires team-composition model. Largest technical risk. Depends on M2 competency.
M4: Gauntlet Master	Future	Full Nuzlocke gauntlet. Depends on M1–M3.

6.6 Additional Thoughts On The Project

6.6.1 Tristan

Something that could probably solve many of the current pain points is a pretraining based on official showdown logs. And then using that pretrained model and finetuning it with RL instead of starting with RL directly. This is something that would have needed more time and was something I was not interested in trying for this project. The problem with pure RL seemed to be a lot more interesting, for us personally.

Overall, looking back that a lot of time was invested in architecture and training set ups that overall did not actually improve anything that much. As such if I could redo the project I would focus more on feature engineering and expanding embedding than looking into LSTM etc. Additionally, while it made sense to me to just play against heuristics, as that would be the opponent in the gauntlet, I would change to just playing against heuristics on a small scale while focusing more on self play in general. Even though the agent wouldn't play against it self in the gauntlet, looking back it would probably stabilise the learning way more than a arbitrary curriculum ever could.

On the topic of curriculum, while I liked working with it in this project it is clear to me now that the curriculum for this project going forward has to be in entropy. With that I mean: new teams for the agent to use, how much info the agent recieves in a given battle etc. instead of opponent AI.

6.6.2 Marvin

My main contribution was the team-pool league curriculum and getting training to actually run stably at scale. The hardware issues we ran into, recurring CUDA illegal memory access errors on the local workstation, were a real time sink. Driver reinstalls, batch size tuning, memory flags: none of it reliably fixed anything. Moving to a cloud instance (vast.ai) with a clean GPU stack was the right call and in hindsight should have been done earlier. The lesson there is pragmatic: if a machine is unstable and you have exhausted the obvious fixes, the environment itself is the problem, not your code.

On the training side, the staged pool curriculum ($3 \to 5 \to 10 \to 20$ teams) versus the fixed 20-team pool was the most interesting result I worked on. The fixed pool plateaued around 30%

win rate vs. heuristic. The staged version climbed to ∼80%. That gap is significant and the reason is fairly intuitive in hindsight: throwing 20 teams at the agent from the start is too much variance too early, and it never gets the chance to solidify fundamentals. Curriculum matters, but the axis of the curriculum matters more than the number of stages.

6.6.3 Team

All in all, we are happy with what we built and genuinely learned a lot, but disappointed with where Milestone 2 landed. The infrastructure is solid, the fixed-team result is strong, and the debugging work we did (reward scaling, self-play, NaN fix, observation pipeline) produced real, measurable improvements at each step. That part we are proud of.

What stings is that none of it translated cleanly to random teams. The same architecture and training setup that achieves 98% vs. heuristic with a fixed team sits at 8% in the random-team setting. That gap is the project's unfinished business, and it is an honest reflection of how much harder generalisation is than memorisation in RL. We did not solve it, and we are not pretending we did.

A. Reproduction Guide

This chapter provides step-by-step instructions for reproducing the training environment, running experiments, and evaluating checkpoints.

A.1 Prerequisites

Requirement	Version / Notes
Python	3.13 (specified in `.python-version`)
`uv`	Latest (package manager, replaces pip)
Node.js	Required for Pokémon Showdown
NVIDIA GPU	Recommended (RTX 3090+ for `standard` preset)
RAM	16 GB minimum, 64 GB recommended for 48 envs

A.2 Installation

```
# Clone the repository
git clone https://github.com/TristanBMuri/DSPR02-FS26-Pokemon_RL.git
cd DSPR02-FS26-Pokemon_RL

# Install dependencies (uses uv, not pip)
uv sync
```

A.3 Starting the Simulator

Pokémon Showdown servers must be running before training. The project provides scripts to manage 8 parallel instances:

```
# Start 8 Showdown servers on ports 8000-8007
./scripts/setup_training.sh

# Stop all servers when done
./scripts/kill_all_showdown.sh
```

A.4 Training Commands

```
# Quick test run (single env, ~150K steps)
uv run train_battler.py --preset quick

# Standard training (48 envs, 10M steps)
uv run train_battler.py --preset standard

# Team-pool league curriculum (production preset, 25M steps typical)
uv run train_battler.py --preset pure_league_play --timesteps 25000000


# Full resources (RTX 5090)
```

```
uv run train_battler.py --preset optimal

# Reduced RAM usage
uv run train_battler.py --preset memory_safe

# Maximum model size
uv run train_battler.py --preset large
```

A.5 Resuming from Checkpoint

Training can be interrupted and resumed without losing progress:

```
# Resume from the latest checkpoint in the same MLflow run
uv run train_battler.py --preset standard \
  --resume-checkpoint latest \
  --mlflow-run-id <RUN_ID>
```

The `<RUN_ID>` can be found in the MLflow UI. If omitted, a new run is created.

A.6 Validation

```
# Validate a specific checkpoint against fixed teams
python scripts/validate_checkpoint.py \
  --checkpoint-path checkpoints/step_XXXX \
  --protocol fixed_paired

# Validate against BDSP trainers (gauntlet data; automation still manual
   )
python scripts/validate_bdsp_trainers.py \
  --checkpoint-path checkpoints/step_XXXX
```

Validation results are logged to MLflow with per-opponent-type breakdowns.

A.7 Linting & Formatting

```
uv run ruff check .
uv run ruff format .
```

A.8 Artifacts

Location	Content
`checkpoints/`	Saved model checkpoints and self-play weights (`selfplay_latest.pt`). Gitignored.
`mlruns/`	MLflow experiment artifacts and metrics.
`data/`	BDSP trainer data, gauntlet order, vocabularies, validation team manifests.
`data/validation/`	Frozen team JSON files for reproducible evaluation.

A.9 Troubleshooting

Issue	Solution
Showdown connection errors	Verify servers are running: `curl localhost:8000`. Restart with the spin-up script.
Out of memory	Reduce `num_workers` or use the `memory_safe` preset. Monitor with `htop`.
Slow training	Check GPU utilisation with `nvidia-smi`. Ensure CUDA is available and the correct preset is selected.
MLflow not logging	Verify `.env` contains valid MLflow credentials. Check that the MLflow server is reachable.

B. Full Observation Space Specification

This appendix provides the complete breakdown of the observation tensor construction. The observation is a dictionary with five keys (Section 3.2), of which `obs` is the largest.

B.1 Token Overview

The `obs` tensor has shape $(13, 164)$: 13 tokens of 164 dimensions each.

Index	Role	Type
0	Global state	Special
1	Our active Pokémon	Pokémon
2–6	Our bench	Pokémon
7	Opp. active Pokémon	Pokémon
8–12	Opp. bench	Pokémon

B.2 Per-Pokémon Token (160 dimensions)

Each Pokémon token (indices 1–12) is structured as follows:

Feature Group	Dims	Encoding	Details
Presence flags	3	Binary	`is_present`, `is_active`, `is_fainted`.
HP fraction	1	Continuous $[0, 1]$	`current_hp / max_hp`.
Base stats	6	Continuous $[0, 1]$	HP, Attack, Defense, Sp. Atk, Sp. Def, Speed normalised by 200.
Types	20	Multi-hot	18 types + 2 padding (one-hot per type slot).
Status condition	7	One-hot	None, Burn, Poison, Toxic, Paralysis, Sleep, Freeze.
Volatile effects	9	Multi-hot	Tracked battle effects (e.g. confusion, flinch, protection).
Stat boosts	7	Continuous $[-1, 1]$	Atk, Def, SpA, SpD, Spe, Accuracy, Evasion. Normalised from $[-6, +6]$.
Item/Ability flags	2	Binary	`has_item`, `has_ability`.
Weight	1	Continuous	Pokémon weight normalised by 100.
Moves	104	Structured	4 moves × 26 features each (see below).

Per-move features (26 dimensions each, 4 moves = 104 dims):

Feature	Dims	Encoding	Details
Move present	1	Binary	1 if move slot is occupied.
Base power	1	Continuous	Normalised by 100. Zero for status moves.
Accuracy	1	Continuous	Raw accuracy (0–1).
Category	3	One-hot	Physical, Special, Status.
Move type	20	One-hot	18 types + 2 padding.

B.3 Global Token (164 dimensions)

Token 0 (the global state token) uses a different layout:

Feature Group	Dims	Encoding	Details
Weather	9	Multi-hot	Deserialized from battle weather dict (after bug fix).
Terrain / Fields	15	Multi-hot	Active terrain and field effects.
Our side conditions	38	Multi-hot	Hazards, screens, tailwind, etc. on the agent's side.
Opp. side conditions	38	Multi-hot	Same for the opponent's side.
Extra features	11	Mixed	See below.
Padding	53	Zeros	Pads to match Pokémon token length (164 dims).

Extra features (11 dimensions):

Feature	Dims	Description
Opponent type	3	One-hot: `opponent_random`, `opponent_heuristic`, `opponent_other`.
Battle turn	1	Normalised: $\min(\text{turn}/100, 1.0)$.
Force switch	1	Binary: agent is forced to switch.
Active trapped	1	Binary: active Pokémon cannot switch.
Available moves	1	Normalised: count/4.0.
Available switches	1	Normalised: count/6.0.
Can Dynamax	1	Binary.
Can Mega Evolve	1	Binary.
Can Z-Move	1	Binary.

B.4 Categorical ID Vectors

Three additional tensors provide integer IDs for vocabulary-embedded entities:

Key	Shape	Range	Padding
`species`	$(13,)$ int32	$[0, 1150]$	0 = absent
`items`	$(13,)$ int32	$[0, 1109]$	0 = absent
`abilities`	$(13,)$ int32	$[0, 215]$	0 = absent

The global token (index 0) always has ID 0 (padding) for all three vectors.

B.5 Action Mask

Key	Shape	Values
`action_mask`	$(14,)$ float32	1.0 = valid, 0.0 = invalid

Mask layout by index:

- Indices 0–3: Regular moves 1–4. Invalid if move slot empty, no PP, or move disabled.
- Indices 4–7: Gimmick-enhanced moves 1–4. Invalid if no gimmick available (no Dynamax, Mega stone, or Z-crystal).
- Indices 8–13: Switch to party slot 1–6. Invalid if the Pokémon is fainted or already active.

B.6 Action Space Mapping

The following table maps each compressed action index to its native `poke-env` resolution:

Index	Category	Compressed Action	Native Resolution
0	Move	Use move 1	`BattleOrder(Move[0])`
1	Move	Use move 2	`BattleOrder(Move[1])`
2	Move	Use move 3	`BattleOrder(Move[2])`
3	Move	Use move 4	`BattleOrder(Move[3])`
4	Gimmick	Gimmick + move 1	First legal: Mega → Z → Dynamax
5	Gimmick	Gimmick + move 2	First legal: Mega → Z → Dynamax
6	Gimmick	Gimmick + move 3	First legal: Mega → Z → Dynamax
7	Gimmick	Gimmick + move 4	First legal: Mega → Z → Dynamax
8	Switch	Switch to slot 1	`BattleOrder(Pokemon[0])`
9	Switch	Switch to slot 2	`BattleOrder(Pokemon[1])`
10	Switch	Switch to slot 3	`BattleOrder(Pokemon[2])`
11	Switch	Switch to slot 4	`BattleOrder(Pokemon[3])`
12	Switch	Switch to slot 5	`BattleOrder(Pokemon[4])`
13	Switch	Switch to slot 6	`BattleOrder(Pokemon[5])`